Assignment 2

Q1)

The query returns 0 because NULL is not a normal value in SQL. The expression: phone_number = NULL doesn't make sense because NULL is unknown and what can be equal to unknow it doesn't make sense, so that expression just kinda don't know what to do because nothing is equal to unknown, the same is true with other aggregate functions too, the correct expression is WHERE phone_number IS NULL; now it checks whether phone number 'IS' NULL not equal to NULL.

Q2)

DISTINCT is redundant in this case because when you are grouping departments (when using GROUP BY) you get only one department per row, DISTINCT eliminates duplicate rows but there are no duplicates coming in this case so DISTINCT is not needed.

Q3)

When they do INNER JOIN between two tables the output contains the rows in which customers have specific orders which they have placed, and as told in question if LEFT JOIN gives more rows than INNER JOIN then there must be some customers in customers table with no orders(NULL), so they do not appear in INNER JOIN but appear in left join creating extra rows.

Q4)

Group by department gives you one row per department and that AVG(SALARY) Command is good because there is one average salary per department but in the given query if each department contains more than one employee name , then how will the query know which name to output in the row corresponding to that department because the whole employee column is selected and nothing is done to it, and I guess if the query is run as it is , then it will output some arbitrary name.

Q5)

Corrected query

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department
HAVING AVG(salary) > 80000;
```

WHERE clause is executed for every row, and HAVING clause is generally used after grouping things

Q6)

a) non-correlated subquery

```
SELECT product_name, price
FROM Product
WHERE price > (SELECT AVG(price) FROM Product);
```

b) correlated subquery

```
SELECT employee_name, department, salary
FROM Employee e
WHERE salary > (
   SELECT AVG(salary)
   FROM Employee
   WHERE department = e.department
);
```

PART – II

---

a)

note this query will return an error because Order is a SQL keyword and it wont register order as a table when I tried to write it I am not changing the name here for the sake of answer.

```
SELECT c.customer_id, c.name, c.email, c.phone
FROM Customer c
WHERE NOT EXISTS (
   SELECT 1
   FROM Order o
   WHERE o.customer_id = c.customer_id
);
```

b)

```
SELECT r.restaurant_id,
       r.name,
       AVG(order_totals.order_value) AS average_order_value
FROM Restaurant r
JOIN (
   SELECT o.order_id,
            o.restaurant_id,
          SUM(mi.price * oi.quantity) AS order_value
   FROM Order o
   JOIN OrderItem oi
       ON o.order_id = oi.order_id
   JOIN MenuItem mi
       ON oi.item_id = mi.item_id
      AND o.restaurant_id = mi.restaurant_id
   GROUP BY o.order_id, o.restaurant_id
) order_totals
   ON r.restaurant_id = order_totals.restaurant_id
GROUP BY r.restaurant_id, r.name
HAVING COUNT(*) > 5;
```

c)

I reused the order-total calculation because both the restaurant average and the platform-wide average depend on complete order values. The subquery in the final WHERE clause is non-correlated because the platform-wide average is one overall value and does not depend on the current restaurant row.

```
WITH order_totals AS (
   SELECT o.order_id,
          o.restaurant_id,
          SUM(mi.price * oi.quantity) AS order_value
   FROM "Order" o
   JOIN OrderItem oi
```

```
      ON o.order_id = oi.order_id
   JOIN MenuItem mi
      ON oi.item_id = mi.item_id
     AND o.restaurant_id = mi.restaurant_id
   GROUP BY o.order_id, o.restaurant_id
),
restaurant_averages AS (
   SELECT r.restaurant_id,
          r.name,
          AVG(ot.order_value) AS average_order_value
   FROM Restaurant r
   JOIN order_totals ot
      ON r.restaurant_id = ot.restaurant_id
   GROUP BY r.restaurant_id, r.name
   HAVING COUNT(*) > 5
)
SELECT restaurant_id,
      name,
      average_order_value
FROM restaurant_averages
WHERE average_order_value > (
   SELECT AVG(order_value)
   FROM order_totals
);
```

d)

I first grouped by restaurant and menu item to compute total quantity ordered, then used RANK() to choose the top item within each restaurant. A basic GROUP BY restaurant_id with MAX(total_quantity) can find the highest quantity, but by itself it does not reliably tell which item produced that maximum, and it also handles ties poorly unless another ranking or join-back step is added.

QUERY :-

```
WITH item_totals AS (
   SELECT mi.restaurant_id,
          mi.item_id,
          mi.item_name,
          SUM(oi.quantity) AS total_quantity
   FROM MenuItem mi
   JOIN OrderItem oi
      ON mi.item_id = oi.item_id
   GROUP BY mi.restaurant_id, mi.item_id, mi.item_name
),
ranked_items AS (
   SELECT restaurant_id,
          item_id,
          item_name,
          total_quantity,
          RANK() OVER (
             PARTITION BY restaurant_id
             ORDER BY total_quantity DESC
          ) AS quantity_rank
   FROM item_totals
)
SELECT r.restaurant_id,
       r.name AS restaurant_name,
       ri.item_id,
       ri.item_name,
       ri.total_quantity
FROM Restaurant r
JOIN ranked_items ri
   ON r.restaurant_id = ri.restaurant_id
WHERE ri.quantity_rank = 1;
```

Part – III

Q1) The functional dependencies

student_id -> student_name

course_id -> course_name, instructor_id

instructor_id -> instructor_name, instructor_office

(student_id, course_id) -> grade, enrollment_date

Q2) Yes the table is 1NF because every column contains single indivisible value, like only one value is present and you cannot divide them further.

Q3)

The table violates **2NF** because some non-key attributes depend on only part of the composite key (student_id, course_id).

Examples of partial dependencies:

student_id -> student_name
course_id -> course_name, instructor_id

These are partial dependencies because student_id alone determines student_name, and course_id alone determines course information, even though the full primary key is (student_id, course_id).

This causes anomalies. For example, if a student enrolls in five courses, the student's name is stored five times. If the student changes their name, every matching enrollment row must be updated; missing one row would create inconsistent data.

Q4)

A transitive dependency exists here:

course_id -> instructor_id
instructor_id -> instructor_name, instructor_office

So we get:

course_id -> instructor_name, instructor_office

This violates **3NF** because instructor details depend on instructor_id, which is not the main key of the original enrollment table. In other words, instructor information is stored through another non-key attribute instead of being stored in a separate instructor table.

This causes anomalies. For example, if an instructor changes offices, every enrollment row for every course taught by that instructor must be updated. If one row is missed, the database may show two different offices for the same instructor.

Q4)

Student(student_id, student_name)

Justification:

student_id -> student_name

Student information is separated because the student name depends only on student_id.

Instructor(instructor_id, instructor_name, instructor_office)

Justification:

instructor_id -> instructor_name, instructor_office

Instructor information is separated because instructor name and office depend only on instructor_id.

Course(course_id, course_name, instructor_id)

Justification:

course_id -> course_name, instructor_id

Course information is separated because each course has one course name and exactly one instructor.

Enrollment(student_id, course_id, grade, enrollment_date)

Justification:

(student_id, course_id) -> grade, enrollment_date

Enrollment keeps only the facts that depend on the combination of student and course. The composite primary key is (student_id, course_id) because a student can take many courses and a course can have many students.

Foreign keys:

Enrollment.student_id -> Student.student_id
Enrollment.course_id -> Course.course_id
Course.instructor_id -> Instructor.instructor_id

PART – IV

For Part II(a), I could have written the “customers who never placed an order” query using either a LEFT JOIN with WHERE o.order_id IS NULL or a NOT EXISTS subquery. I chose NOT EXISTS because it directly expresses the idea of “there is no matching order for this customer.” It is also safer conceptually because the goal is absence, not combining rows. A careless join can accidentally create duplicate customer rows when there are multiple matching orders, while NOT EXISTS simply tests whether any related row exists.

In Part III, normalizing the Enrollment table into 3NF reduces redundancy and prevents update, insertion, and deletion anomalies. However, it trades off against query convenience. After splitting the table into Student, Course, Instructor, and Enrollment, a query showing a student’s name, course name, instructor office, and grade now requires several joins.